

Sanity Document Retention — Findings

Project v3sfsmlld · dataset **production** · caterflow-by-synapse · as of 21 Sept 2026

Summary: the retention design and deletion logic are correct and verified. The reason old records are still present is operational, not a design flaw: the automated cleanup pipeline stopped completing runs on **9 September 2026** and a run started on **20 September** is stuck mid-way through, with no progress since. As a direct result, **6,799 documents (22% of the dataset)** that are already past their 90-day retention cutoff and eligible for deletion are still sitting in Sanity. This report documents the evidence for both halves of that finding.

TOTAL DOCUMENTS

31,059

OVERDUE & DELETE-ELIGIBLE

6,799 (22%)

DAYS SINCE LAST DELETE

12 days

A note on the 25,000-document figure

An earlier version of this analysis was built around a 25,000-document quota. That number does not match any documented Sanity plan tier and was not independently verified before use. Sanity's actual published tiers (sanity.io/docs/content-lake/technical-limits) are **10,000 documents on the Free plan, 50,000 on Growth, and 1,000,000+ on Enterprise**. Since this project holds 31,059 documents and is still actively writing new ones without being blocked, it cannot be on the Free plan (Sanity blocks document creation once that cap is hit) — it must already be on Growth or higher, which would put current usage at roughly **62% of a 50,000 quota, not over any real limit**. The exact plan isn't visible via the API token used for this analysis; it should be confirmed on manage.sanity.io before this figure is quoted anywhere else. The retention findings below hold regardless of which tier applies.

Evidence the retention design itself is correct

- **No orphaned documents anywhere.** Every reference field across the entire dataset was scanned — all 31,059 documents, ~510,000 individual reference fields — and zero point at a non-existent document. Sanity's strong-reference model blocks any delete that would create a dangling reference, and no field in this schema is configured as a weak reference, so this is structurally guaranteed, not just observed.
- **File cleanup is complete and correct.** Every FileAttachment record's parent reference resolves to a live document, and every underlying file asset (image/PDF) is referenced by exactly one FileAttachment — no orphaned metadata records, no unused binary assets left behind.
- **The deletion order is correct.** Collections are processed child-before-parent (FileAttachment/StockSnapshot before GoodsReceipt/DispatchLog/PurchaseOrder/etc.), and a previously identified stale-reference bug (StockSnapshot.lastTransaction blocking parent deletes) was already fixed and verified — zero StockSnapshot documents remain in the dataset today.
- **Reference-order "errors" in the run logs are expected, not bugs.** The cleanup logs show cases where a PurchaseOrder can't be deleted yet because the GoodsReceipt that fulfilled it hasn't individually crossed its own retention cutoff. This is intentional: each document type is evaluated against its own business date, so a PO and its receipt routinely age out at different times. It resolves on its own as long as cleanup keeps running.

What's actually wrong: the pipeline stopped running

Checked directly against the production archive database (MongoDB). The scheduled jobs (daily cleanup + a 5-minute resume check, configured in `vercel.json`) have not produced a single additional deletion since 9

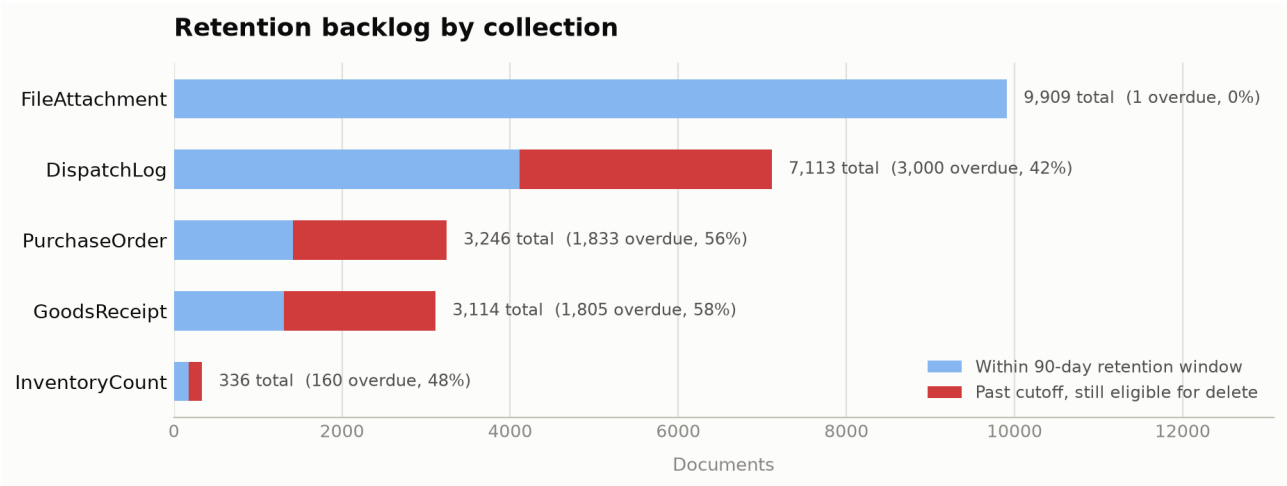
September.



The run that started on 20 September completed its Sanity-archiving steps for DispatchLogs, PurchaseOrders, GoodsReceipts, InternalTransfers and StockAdjustments without issue, then stalled partway into the InventoryCounts step and has not logged any progress since — consistent with a database connection timing out mid-run rather than a code error (no exception was recorded). While that run's status remains "in progress," the system won't safely start a new one, which is why no further cleanup has happened in the 12 days since, even though the underlying deletion logic is ready to run.

Retention backlog by collection

The direct consequence: documents that should already be gone under the existing 90-day retention policy are still present. This is a snapshot of what's overdue right now — not a policy gap, a backlog waiting on the pipeline.



Date range detail

Collection	Total docs	Oldest record	Newest record	Overdue & delete-eligible	% overdue
DispatchLog	7,113	2025-12-08	2026-09-19	3,000	42%
GoodsReceipt	3,114	2025-12-10	2026-09-19	1,805	58%
PurchaseOrder	3,246	2025-12-10	2026-09-19	1,833	56%
InventoryCount	336	2025-12-08	2026-09-19	160	48%
FileAttachment	9,909	2026-03-12	2026-09-19	1	0%
TOTAL	23,718			6,799	29%

"Overdue & delete-eligible" counts only records in a finished workflow status (completed/cancelled/adjusted) whose business date is older than the 90-day cutoff (2026-06-23) — i.e. records the system's own rules say are safe to remove. It excludes older records still in draft/pending status, which are correctly retained regardless of age. One DispatchLog record carries a

corrupted future date (a data-entry error, not a retention issue) and is excluded from the date-range figures above.

Recommended next step

Clear the stalled run (a dedicated endpoint exists for this: *POST /api/archive/lock/clear*) and confirm the next scheduled run reaches completion rather than stalling again. If it stalls again on a different collection, that points to the archive database's connection limits rather than anything in the retention logic itself. Once runs are completing normally, this backlog should clear within a few days without any further manual intervention.